



UNIVERSIDADE FEDERAL DE SANTA CATARINA
CAMPUS ARARANGUÁ
CURSO DE ENGENHARIA DA COMPUTAÇÃO

Felipe Luan Schmitt

Trabalho 3: Cronômetro de Basquete

Disciplina: Linguagem de Descrição de Hardware

LISTA DE FIGURAS

Figura 1 – Diagrama de estados do cronômetro de basquete.	4
Figura 2 – Formas de onda do divisor de clock.	8
Figura 3 – Formas de onda da simulação dos contadores.	23
Figura 4 – Formas de onda da FSM.	29
Figura 5 – Formas de onda do módulo independente de plataforma.	37
Figura 6 – Formas de onda do módulo de topo e interfaces.	45
Figura 7 – Diagrama de pinos de periféricos da Nexys 1.	47

SUMÁRIO

1	INTRODUÇÃO	3
2	DESENVOLVIMENTO	4
2.1	MÓDULOS INDIVIDUAIS	5
2.1.1	Divisor de clock	5
2.1.1.1	Implementação	5
2.1.1.2	Validação	6
2.1.2	Contador de centésimos	8
2.1.2.1	Implementação	8
2.1.3	Contador de segundos	10
2.1.3.1	Implementação	10
2.1.4	Contador de minutos	13
2.1.4.1	Implementação	13
2.1.5	Contador de quartos	16
2.1.5.1	Implementação	16
2.1.6	Validação dos contadores	18
2.1.7	Máquina de Estados Finitos (FSM)	23
2.1.7.1	Implementação	23
2.1.7.2	Validação	26
2.1.8	Módulo independente de plataforma	29
2.1.8.1	Implementação	29
2.1.8.2	Validação	33
2.1.9	Módulo de topo (Top-Level)	37
2.1.9.1	Implementação	37
2.1.9.2	Validação	41
2.2	PROTOTIPAÇÃO NA PLACA NEXYS 1	45
2.2.1	Implementação do mapeamento	45
2.2.2	Vídeo de demonstração	48
3	CONCLUSÃO	49

1 INTRODUÇÃO

Este documento apresenta o desenvolvimento e a implementação de um cronômetro para jogos de basquete em hardware, requisitado como o Trabalho T3 da disciplina de Linguagem de Descrição de Hardware. O objetivo principal do projeto consiste em descrever o circuito utilizando a linguagem VHDL, realizar a sua validação por meio de simulações com *testbenches* e, por fim, prototipar o sistema validado em uma placa de prototipação. O cronômetro projetado opera com precisão de um centésimo de segundo, permitindo a configuração de um valor inicial de tempo e de quarto, operando nos modos FIBA ou NBA, e executando a contagem regressiva até que o tempo se esgote.

A placa de desenvolvimento alvo para a prototipação é a família Digilent Nexys 1. Para a interface com o usuário, o projeto faz uso exclusivo dos periféricos nativos oferecidos pela placa, o que inclui oito chaves deslizantes para a inserção dos dados de carga de tempo e de quarto, quatro botões do tipo *push-button* para a emissão de comandos de controle, oito diodos emissores de luz (LEDs) para a sinalização do quarto atual e dos minutos restantes, e quatro mostradores (*displays*) de sete segmentos multiplexados para a exibição visual dos segundos e centésimos. O sincronismo do circuito lógico é garantido pela utilização da fonte de clock de 50 MHz contida na própria placa, através de um cristal.

O desenvolvimento da arquitetura de hardware seguiu a metodologia de design hierárquico, particionando o escopo do problema em submódulos funcionais independentes. A lógica central do projeto foi encapsulada em um módulo independente de plataforma, responsável por coordenar os contadores de tempo e abrigar a Máquina de Estados Finitos (FSM, *Finite-State Machine*) que transita o sistema entre os estados de repouso, carregamento de dados e contagem ativa. Posteriormente, este módulo independente foi instanciado no módulo de topo da hierarquia, o qual integra as partes dependentes da placa para a interface física. Cada bloco lógico gerado foi desenvolvido e testado isoladamente, garantindo que a lógica de funcionamento esteja correta antes da integração completa no projeto.

Neste capítulo, primeiramente serão abordados os módulos de forma individual, abrangendo o divisor de *clock*, os cinco contadores responsáveis pelas parcelas de tempo e quartos e a FSM, acompanhados de demonstrações do código, *testbenches* e formas de onda. Cada um dos contadores será responsável por uma grandeza de tempo, sendo o divisor de clock para converter os clocks para centésimos, os outros para centésimos, segundos, minutos e quartos. Posteriormente, será demonstrada a integração da lógica no módulo independente de plataforma e, em seguida, a montagem final no arquivo topo da hierarquia, responsável pelos filtros de botões e conversores de *display*. Por fim, haverá a demonstração da prototipação física na placa Nexys.

2.1 MÓDULOS INDIVIDUAIS

2.1.1 Divisor de clock

O responsável por gerar os pulsos de clock é o crista de quartzo presente na placa Nexys 1, nela é possível escolher as frequências 25, 50 e 100 MHz através de um jumper, para o caso do presente trabalho será utilizado o clock de 50 MHz.

Para converter os 50MHz para centésimos, foi desenvolvido um divisor de clock, que funcionará como um contador em que a cada 500000 pulsos de clock, contará 1 centésimo.

2.1.1.1 Implementação

Segue abaixo a entidade do módulo.

Entity de divisor_clock.vhd

```
5 ENTITY divisor_clock IS
6     GENERIC (
7         CLOCK_FREQ: integer := 500000
8     );
9     PORT (
10        clock: IN std_logic;
11        reset: IN std_logic;
12        passou_1cent: OUT std_logic
13    );
14 END ENTITY divisor_clock;
```

Fonte: Do autor (2026)

Foi utilizado na entrada o *clock* e o *reset*, e a saída é *passou_1cent*, já o *CLOCK_FREQ* é o valor a qual o clock será dividido para a conversão para centésimo.

Abaixo está presente a *ARCHITECTURE* do divisor de clock, responsável pela parte lógica de seu funcionamento. Foi gerado um sinal interno chamado *contador*, de tamanho 0 a *CLOCK_FREQ*, ou seja, 500000, que será utilizado na contagem de pulsos. Foi utilizado um *PROCESS* com *clock*

e `reset` na lista de sensibilidade, dentro dele, a primeira condição é relacionado ao `reset`, isso significa que ele funciona de forma assíncrona, sem depender dos pulsos de clock.

A segunda condição refere-se à borda de subida do clock, cada vez que ela ocorre, é verificado se o contador já atingiu o valor igual à `CLOCK_FREQ - 1`, isso porque o intervalo de contagem vai de 0 a 499999. Se for válida, o `passou_1cent` dará um pulso para '1' durante um período de clock, sinalizando que passou exatamente 1 centésimo, se não, o contador incrementará em 1 no seu valor até que a condição anterior seja satisfeita.

Architecture de divisor_clock.vhd

```
16 ARCHITECTURE behavior OF divisor_clock IS
17     SIGNAL contador: integer RANGE 0 TO CLOCK_FREQ := 0;
18 BEGIN
19     PROCESS(clock, reset)
20     BEGIN
21         IF reset = '1' THEN
22             contador <= 0;
23             passou_1cent <= '0';
24         ELSIF rising_edge(clock) THEN
25             IF contador = (CLOCK_FREQ - 1) THEN
26                 contador <= 0;
27                 passou_1cent <= '1';
28             ELSE
29                 contador <= contador + 1;
30                 passou_1cent <= '0';
31             END IF;
32         END IF;
33     END PROCESS;
34 END behavior;
```

Fonte: Do autor (2026)

2.1.1.2 Validação

Para validação foi desenvolvido um *testbench* que testa a divisão do clock. Nele são gerados os sinais para cada uma das entradas e saída do módulo e utilizado um `CLOCK_FREQ` com valor 4 para melhor visualização nas formas de onda. Após o `BEGIN`, é instanciado o submódulo do `divisor_clock.vhd` e atribuídos os sinais internos do *testbench* às portas do submódulo.

Posteriormente é utilizado um `PROCESS`, onde o `clock` alterna de '0' para '1' e vice-versa a cada 10 ns. Já o `reset` começa em '1' e após 40 ns passa para '0' e o divisor passa a operar corretamente.

Segue abaixo o código desenvolvido, considere que, no relatório, estão sendo apresentadas somente as partes funcionais do código, deixando de lado as bibliotecas utilizadas e a entidade vazia.

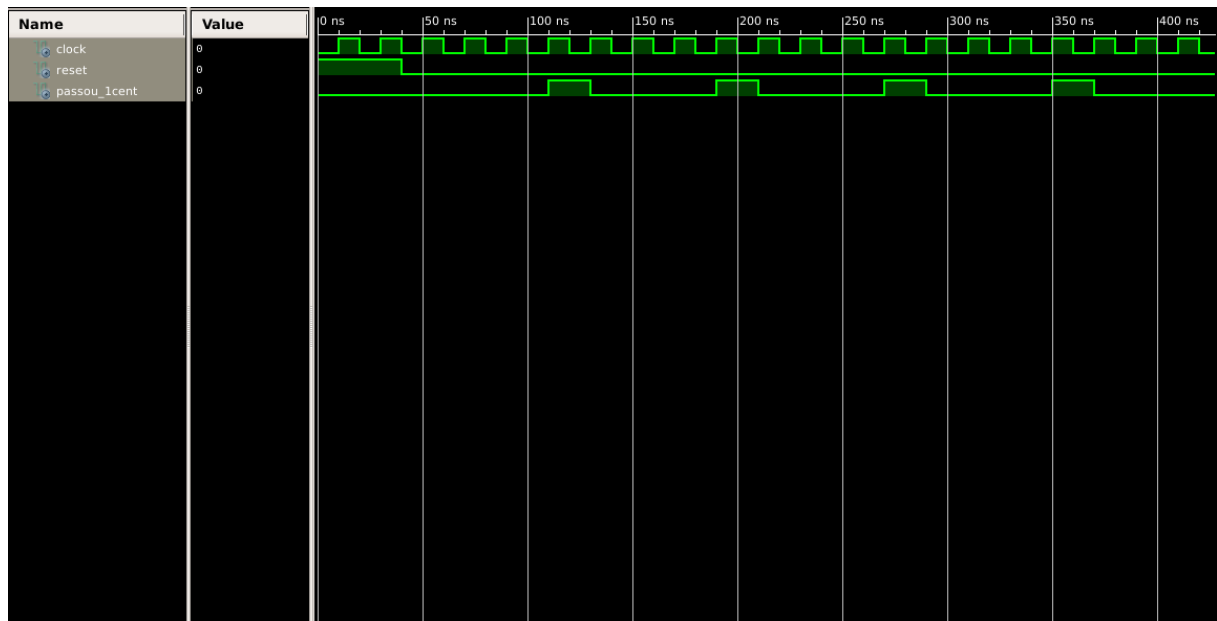
Architecture de divisor_clock.vhd

```
7 ARCHITECTURE behavior OF tb_divisor_clock IS
8     SIGNAL clock: std_logic := '0';
9     SIGNAL reset: std_logic := '0';
10    SIGNAL passou_1cent: std_logic;
11 BEGIN
12     UUT: ENTITY work.divisor_clock
13         GENERIC MAP(
14             CLOCK_FREQ => 4
15         )
16         PORT MAP(
17             clock => clock,
18             reset => reset,
19             passou_1cent => passou_1cent
20         );
21     clock_process: PROCESS
22     BEGIN
23         clock <= '0';
24         WAIT FOR 10 ns;
25         clock <= '1';
26         WAIT FOR 10 ns;
27     END PROCESS;
28     reset_process: PROCESS
29     BEGIN
30         reset <= '1';
31         WAIT FOR 40 ns;
32         reset <= '0';
33         WAIT;
34     END PROCESS;
35 END behavior;
```

Fonte: Do autor (2026)

Abaixo é apresentada uma imagem referente às ondas geradas pelo *testbench* implementado. É possível notar que seu funcionamento está correto, onde a cada 4 bordas de subida de clock, é realizado um pulso, em que no caso da implementação real com o `CLOCK_FREQ = 500000`, esse pulso avisa que passou exatamente 1 centésimo.

Figura 2 – Formas de onda do divisor de clock.



Fonte: Do autor (2026).

2.1.2 Contador de centésimos

O responsável por registrar e decrementar os centésimos de segundo do cronômetro é o módulo contador de centésimos.

Para realizar esta contagem, foi desenvolvido um contador decrescente de módulo 100, que funcionará variando de 99 a 0 de forma circular, sendo habilitado a cada pulso proveniente do divisor de clock.

2.1.2.1 Implementação

Segue abaixo a entidade do módulo.

Entity de cont_cen.vhd

```

5 ENTITY cont_cen IS
6   PORT(
7     clock: IN std_logic;
8     reset: IN std_logic;
9     fim_quarto: IN std_logic;
10    EA: IN std_logic_vector(1 DOWNTO 0);
11    passou_1cent: IN std_logic;
12    modo_novoquarto: IN std_logic;
13    carga: IN std_logic;
14    centesimos: OUT std_logic_vector(6 DOWNTO 0);
15    passou_1seg: OUT std_logic;
16    fim_jogo: IN std_logic

```

```

17     );
18 END ENTITY cont_cent;

```

Fonte: Do autor (2026)

Foi utilizado na entrada o clock, reset, fim_quarto, EA, passou_1cent, modo_novoquarto, carga e fim_jogo, e a saída é o vetor centesimos e o pulso passou_1seg.

Abaixo está presente a ARCHITECTURE do contador de centésimos, responsável pela parte lógica do seu funcionamento. Foi gerado um sinal interno chamado conta_cent, de tamanho 0 a 99, que será utilizado na simplificação das operações matemáticas da contagem. Foi utilizado um PROCESS com clock e reset na lista de sensibilidade, dentro dele, a primeira condição é relacionada ao reset, isso significa que ele funciona de forma assíncrona, sem depender dos pulsos de clock.

A segunda condição refere-se à borda de subida do clock, cada vez que ela ocorre, é verificado inicialmente se o fim_jogo é '0', para garantir que a máquina obedece à trava de segurança. Estando o jogo a decorrer, o código avalia os comandos de controle: se a carga for '1' nos estados "01" ou "11", ou se o modo_novoquarto for '1' no estado "11", o contador é forçado a zero, visto que a carga manual afeta apenas as outras parcelas de tempo.

A lógica de decréscimo apenas ocorre se o estado da máquina (EA) for "10", se o passou_1cent for '1' e o fim_quarto for '0'. Se for válida e o contador estiver em 0, o conta_cent passará para 99, fazendo a virada, se não, o contador decrementará em 1 no seu valor. Fora do processo síncrono, a contagem é convertida para std_logic_vector na saída principal, e o passou_1seg dará um pulso para '1' gerado de forma no exato instante em que o contador atinge 0, eliminando atrasos indesejados.

Architecture de cont_cen.vhd

```

20 ARCHITECTURE behavior OF cont_cent IS
21     SIGNAL conta_cent: integer RANGE 0 TO 99 := 0;
22 BEGIN
23     PROCESS(clock, reset)
24     BEGIN
25         IF reset = '1' THEN
26             conta_cent <= 0;
27         ELSIF rising_edge(clock) THEN
28             IF fim_jogo = '0' THEN
29                 IF carga = '1' AND (EA = "01" OR EA = "11") THEN
30                     conta_cent <= 0;
31                 ELSIF EA = "11" THEN
32                     IF modo_novoquarto = '1' THEN
33                         conta_cent <= 0;
34                     END IF;
35                 ELSIF EA = "10" THEN
36                     IF passou_1cent = '1' AND fim_quarto = '0' THEN

```

```

37         IF conta_cent = 0 THEN
38             conta_cent <= 99;
39         ELSE
40             conta_cent <= conta_cent - 1;
41         END IF;
42     END IF;
43 END IF;
44 END IF;
45 END IF;
46 END PROCESS;
47 centesimos <= std_logic_vector(to_unsigned(conta_cent, 7));
48 passou_1seg <= '1' WHEN (conta_cent = 0 AND passou_1cent = '1' AND
    EA = "10") ELSE '0';
49 END behavior;

```

Fonte: Do autor (2026)

2.1.3 Contador de segundos

O módulo contador de segundos é o responsável por registrar e decrementar a segunda parcela de tempo do cronômetro. O seu funcionamento é baseado num contador decrescente de módulo 60, que varia de 59 a 0 de forma circular. A habilitação da sua contagem está diretamente atrelada ao pulso de *enable* gerado pelo contador de centésimos, garantindo o sincronismo perfeito da cascata de tempo.

2.1.3.1 Implementação

Segue abaixo a entidade do módulo, que define as interfaces físicas e lógicas de comunicação.

Entity de cont_seg.vhd

```

5 ENTITY cont_seg IS
6     PORT(
7         clock: IN std_logic;
8         reset: IN std_logic;
9         fim_quarto: IN std_logic;
10        EA: IN std_logic_vector(1 DOWNTO 0);
11        passou_1seg: IN std_logic;
12        modo_novoquarto: IN std_logic;
13        carga: IN std_logic;
14        c_seg: IN std_logic_vector(1 DOWNTO 0);
15        segundos: OUT std_logic_vector(5 DOWNTO 0);
16        passou_1min: OUT std_logic;
17        fim_jogo: IN std_logic
18    );

```

```
19 END ENTITY cont_seg;
```

Fonte: Do autor (2026)

Nesta estrutura, as entradas são quase idênticas às do módulo anterior, acrescentando-se o vetor `c_seg` (proveniente das chaves físicas da placa) e o sinal `passou_1seg` (que atua como permissão de contagem). Como saídas, o módulo exporta o vetor `segundos` de 6 bits e gera o pulso combinacional `passou_1min`, que será utilizado para habilitar o decremento no contador de minutos subsequente.

Na ARCHITECTURE, foi declarado o sinal interno `conta_seg` limitado de 0 a 59. A lógica síncrona dentro do PROCESS inicia avaliando o `reset` assíncrono, que zera o contador imediatamente, independente do `clock`.

Início da Architecture e Reset

```
21 ARCHITECTURE behavior OF cont_seg IS
22     SIGNAL conta_seg: integer RANGE 0 TO 59 := 0;
23 BEGIN
24     PROCESS(clock, reset)
25     BEGIN
26         IF reset = '1' THEN
27             conta_seg <= 0;
28         ELSIF rising_edge(clock) THEN
```

Fonte: Do autor (2026)

Com a ocorrência da borda de subida do relógio, o código verifica inicialmente se o jogo ainda está em andamento (`fim_jogo = '0'`). Confirmada esta condição, o sistema passa a avaliar a função de carregamento manual de tempo. Para que o utilizador consiga inserir um novo valor nos segundos, é obrigatório que o botão de carga esteja ativado e que o cronómetro esteja num estado de pausa, ou seja, nos estados IDLE ("01") ou STOPPED ("11").

Neste ponto, surge uma limitação de hardware que molda a escrita do código: a placa Nexys possui apenas oito chaves deslizantes no total. Como seis destas chaves já estão comprometidas para a definição dos quartos e dos minutos, restam apenas duas chaves (representadas pelo vetor `c_seg`) para configurar os segundos. Duas chaves geram apenas quatro combinações lógicas possíveis ("00", "01", "10" e "11"), tornando impossível representar todos os números de 0 a 59 de forma direta.

Para contornar este problema, a especificação do projeto define que a leitura destas duas chaves deve ser interpretada em frações de um quarto de minuto, multiplicando as entradas por incrementos exatos de 15 segundos. Dessa forma, a lógica interna do código faz a seguinte conversão: a combinação "00" define o tempo em 0 segundos, "01" define em 15 segundos, "10" ajusta para 30 segundos e "11" crava o valor em 45 segundos.

Abaixo desta verificação de carga, existe também a lógica para a transição de período. Se

o utilizador não acionou a carga, mas o cronómetro está parado ("11") e o botão `modo_novoquarto` é pressionado, o contador de segundos é forçado a retornar a 0, preparando automaticamente o painel para o início do próximo quarto.

Lógica de Carga e Novo Quarto

```

29         IF fim_jogo = '0' THEN
30             IF carga = '1' AND (EA = "01" OR EA = "11") THEN
31                 IF c_seg = "00" THEN
32                     conta_seg <= 0;
33                 ELSIF c_seg = "01" THEN
34                     conta_seg <= 15;
35                 ELSIF c_seg = "10" THEN
36                     conta_seg <= 30;
37                 ELSIF c_seg = "11" THEN
38                     conta_seg <= 45;
39                 END IF;
40             ELSIF EA = "11" THEN
41                 IF modo_novoquarto = '1' THEN
42                     conta_seg <= 0;
43                 END IF;

```

Fonte: Do autor (2026)

A etapa de contagem decrescente possui a mesma lógica do módulo de centésimos. O tempo conta quando a máquina se encontra no estado COUNTING ("10"), o pulso `passou_1seg` permite a ação e o `fim_quarto` não foi alcançado. Ao atingir zero, o contador realiza a virada para 59.

Lógica de Decréscimo

```

44         ELSIF EA = "10" THEN
45             IF passou_1seg = '1' AND fim_quarto = '0' THEN
46                 IF conta_seg = 0 THEN
47                     conta_seg <= 59;
48                 ELSE
49                     conta_seg <= conta_seg - 1;
50                 END IF;
51             END IF;
52         END IF;
53     END IF;
54 END IF;
55 END PROCESS;

```

Fonte: Do autor (2026)

Para concluir o módulo, foram declaradas as atribuições combinacionais fora do bloco do processo. O valor inteiro `conta_seg` é convertido para o vetor lógico de saída `segundos`. Em

simultâneo, o sinal `passou_1min` emite um pulso de *enable* apenas no exato instante em que o contador de segundos marca zero, o aviso do contador anterior (`passou_1seg`) chega e a contagem está ativa (`EA = "10"`). Esta construção concorrente simula o comportamento de portas lógicas externas, eliminando a formação indevida de registradores e atrasos na cascata de tempo.

Atribuições Combinacionais de Saída

```

56     segundos <= std_logic_vector(to_unsigned(conta_seg, 6));
57     passou_1min <= '1' WHEN (conta_seg = 0 AND passou_1seg = '1' AND EA
    = "10") ELSE '0';
58 END behavior;
```

Fonte: Do autor (2026)

2.1.4 Contador de minutos

O contador de minutos é a última etapa da contagem de tempo. Diferente dos centésimos e segundos, que funcionam de forma circular e voltam ao valor máximo, este módulo para ao chegar em zero, marcando o fim do período de jogo. Além disso, a sua lógica foi construída para funcionar com as regras de duas federações diferentes: a FIBA, com quartos de 10 minutos, e a NBA, com quartos de 12 minutos.

2.1.4.1 Implementação

Abaixo é apresentada a entidade do módulo, que agora inclui os sinais necessários para identificar a regra escolhida e receber o valor das chaves da placa.

Entity de cont_min.vhd

```

5 ENTITY cont_min IS
6     PORT(
7         clock: IN std_logic;
8         reset: IN std_logic;
9         EA: IN std_logic_vector(1 DOWNT0 0);
10        passou_1min: IN std_logic;
11        modo_novoquarto: IN std_logic;
12        modo_nba: IN std_logic;
13        carga: IN std_logic;
14        c_min: IN std_logic_vector(3 DOWNT0 0);
15        minutos: OUT std_logic_vector(3 DOWNT0 0);
16        fim_quarto: IN std_logic;
17        fim_jogo: IN std_logic
18    );
19 END ENTITY cont_min;
```

Fonte: Do autor (2026)

Nesta parte, a principal novidade é a entrada `modo_nba`, que avisa ao módulo qual a duração máxima do quarto a ser usada. Outra diferença está no vetor `c_min` e na saída `minutos`, ambos com 4 bits, tamanho que permite representar números até 15 em binário puro. A redução do contador agora é liberada pelo pulso de entrada `passou_1min`.

Na ARCHITECTURE, foi criado o sinal `conta_min` com limite até 12. O processo principal tem um detalhe importante na sua lista de sensibilidade: a presença do sinal `modo_nba`. Isso garante que, na hora do *reset* assíncrono, o simulador perceba na mesma hora se a chave de modo mudou. Se a regra for da NBA, o cronômetro começa com 12; se não, segue o padrão FIBA de 10 minutos.

Início da Architecture e Lógica de Reset

```

21 ARCHITECTURE behavior OF cont_min IS
22     SIGNAL conta_min: integer RANGE 0 TO 12 := 12;
23 BEGIN
24     PROCESS(clock, reset, modo_nba)
25     BEGIN
26         IF reset = '1' THEN
27             IF modo_nba = '1' THEN
28                 conta_min <= 12;
29             ELSE
30                 conta_min <= 10;
31             END IF;
32         ELSIF rising_edge(clock) THEN

```

Fonte: Do autor (2026)

Avançando para as operações com o *clock*, o código tem uma proteção de *hardware* importante para o carregamento manual. Como a placa possui quatro chaves para os minutos, o usuário poderia tentar colocar valores de 0 a 15. Porém, o máximo que o sinal interno aceita é 12. Para evitar erros no circuito, foi colocada uma trava: se o valor lido for maior que o máximo de minutos, o contador assume o teto máximo de 10 ou 12, dependendo do modo que está.

Além disso, esta parte do código faz a troca de quarto. Quando o botão `modo_novoquarto` é apertado com o cronômetro pausado ("11"), o contador volta para 10 ou 12 minutos, dependendo do modo de jogo escolhido.

Proteção de Carga e Novo Quarto

```

33     IF fim_jogo = '0' THEN
34         IF carga = '1' AND (EA = "01" OR EA = "11") THEN
35             IF modo_nba = '1' THEN
36                 IF to_integer(unsigned(c_min)) > 12 THEN
37                     conta_min <= 12;
38                 ELSE
39                     conta_min <= to_integer(unsigned(c_min));
40                 END IF;

```

```

41         ELSE
42             IF to_integer(unsigned(c_min)) > 10 THEN
43                 conta_min <= 10;
44             ELSE
45                 conta_min <= to_integer(unsigned(c_min));
46             END IF;
47         END IF;
48     ELSIF EA = "11" THEN
49         IF modo_novoquarto = '1' THEN
50             IF modo_nba = '1' THEN
51                 conta_min <= 12;
52             ELSE
53                 conta_min <= 10;
54             END IF;
55         END IF;

```

Fonte: Do autor (2026)

Por fim, acontece a contagem decrescente. Como os minutos são a maior unidade de tempo do cronômetro, a redução ativada pelo `passou_1min` só diminui o valor atual se o contador for maior que zero (`conta_min > 0`). Ao chegar em zero, a contagem apenas para.

No final do processo, o valor é convertido e enviado para a saída `minutos`. Diferente dos submódulos anteriores, o contador de minutos não precisa criar um sinal extra de aviso, já que a condição de fim de quarto é verificada no módulo principal apenas quando os três tempos chegam a zero.

Lógica de Decréscimo e Saída

```

48     ELSIF EA = "10" THEN
49         IF passou_1min = '1' AND fim_quarto = '0' THEN
50             IF conta_min > 0 THEN
51                 conta_min <= conta_min - 1;
52             END IF;
53         END IF;
54     END IF;
55 END IF;
56 END IF;
57 END PROCESS;
58 minutos <= std_logic_vector(to_unsigned(conta_min, 4));
59 END behavior;

```

Fonte: Do autor (2026)

2.1.5 Contador de quartos

O contador de quartos é o módulo responsável por registrar o período atual do jogo. Diferente dos contadores de tempo que operam diminuindo os valores, este módulo funciona de forma crescente, avançando do primeiro ao quarto período da partida. Para deixar o circuito mais simples, a contagem interna utiliza apenas dois bits, variando de 0 a 3, o que representa perfeitamente os quatro quartos do jogo (sendo 0 o Q1 e 3 o Q4).

2.1.5.1 Implementação

Abaixo é apresentada a entidade do módulo, que recebe os comandos de controle e a posição das chaves.

Entity de cont_quarto.vhd

```

5 ENTITY cont_quarto IS
6     PORT(
7         clock: IN std_logic;
8         reset: IN std_logic;
9         EA: IN std_logic_vector(1 DOWNTO 0);
10        carga: IN std_logic;
11        modo_novoquarto: IN std_logic;
12        c_quarto: IN std_logic_vector(1 DOWNTO 0);
13        quarto: OUT std_logic_vector(1 DOWNTO 0);
14        fim_jogo: IN std_logic
15    );
16 END ENTITY cont_quarto;
```

Fonte: Do autor (2026)

Nesta parte, as entradas incluem o vetor `c_quarto` de 2 bits, que vem das chaves físicas da placa, e a saída `quarto`, também de 2 bits. Essa saída será enviada para o módulo principal, que fará a tradução desse valor para acender os LEDs correspondentes no painel. O módulo também recebe o aviso `fim_jogo` para bloquear alterações indevidas quando a partida acaba.

Na ARCHITECTURE, foi criado o sinal `conta_quarto` com limite de 0 a 3. O processo principal inicia verificando o *reset* assíncrono. Quando o sistema é reiniciado, o contador volta imediatamente para 0, garantindo que o cronômetro comece sempre no primeiro quarto (Q1).

Início da Architecture e Lógica de Reset

```

18 ARCHITECTURE behavior OF cont_quarto IS
19     SIGNAL conta_quarto: integer RANGE 0 TO 3 := 0;
20 BEGIN
21     PROCESS(clock, reset)
22     BEGIN
23         IF reset = '1' THEN
24             conta_quarto <= 0;
```

```
25      ELSIF rising_edge(clock) THEN
```

Fonte: Do autor (2026)

Avançando para as operações ativadas pelo *clock*, o código confere primeiro se o jogo ainda está a decorrer através da trava *fim_jogo*. Se a partida não acabou, o sistema aceita a mudança de período. Caso o botão de carga seja apertado com o cronômetro pausado (estados IDLE ou STOPPED), o contador assume o valor exato que o usuário configurou nas chaves.

Além da carga manual, esta seção cuida do avanço natural do jogo. Quando o tempo para após o fim de um período (estado "11") e o botão *modo_novoquarto* é pressionado, o módulo soma 1 ao quarto atual. Para evitar problemas na contagem, existe uma proteção lógica: a soma só acontece se o contador for menor que 3. Isso garante que o botão perca o efeito no final do quarto período (Q4), respeitando as regras especificadas para o esporte.

Lógica de Carga e Avanço de Quarto

```
26      IF fim_jogo = '0' THEN
27          IF carga = '1' AND (EA = "01" OR EA = "11") THEN
28              conta_quarto <= to_integer(unsigned(c_quarto));
29          ELSIF EA = "11" THEN
30              IF modo_novoquarto = '1' THEN
31                  IF conta_quarto < 3 THEN
32                      conta_quarto <= conta_quarto + 1;
33                  END IF;
34              END IF;
35          END IF;
36      END IF;
37  END IF;
38  END PROCESS;
```

Fonte: Do autor (2026)

No final do código, fora do processo síncrono, o valor numérico interno é transformado de volta para vetor lógico e entregue à saída *quarto*. Diferente dos módulos de minutos e segundos, este bloco não possui lógica de contagem contínua movida pelo relógio, pois ele reage apenas aos comandos diretos feitos pelo usuário.

Saída do Quarto Atual

```
39      quarto <= std_logic_vector(to_unsigned(conta_quarto, 2));
40  END behavior;
```

Fonte: Do autor (2026)

2.1.6 Validação dos contadores

Para a validação do fluxo de dados (*datapath*), foi desenvolvido um *testbench* que integra e testa todos os contadores do cronômetro em conjunto. Nele, são gerados os sinais para simular o comportamento da Máquina de Estados Finitos e as ações do utilizador. Assim como no teste individual, o parâmetro `CLOCK_FREQ` do divisor de *clock* foi ajustado para o valor 4, permitindo uma melhor visualização do decréscimo de tempo nas formas de onda.

Após o `BEGIN`, são instanciados os cinco submódulos desenvolvidos (`divisor_clock`, `cont_cent`, `cont_seg`, `cont_min` e `cont_quarto`) e atribuídos os sinais internos do *testbench* às suas respectivas portas. Logo em seguida, foram adicionadas atribuições combinacionais para gerar os avisos de `fim_quarto` (quando os minutos, segundos e centésimos chegam simultaneamente a zero) e de `fim_jogo` (quando o quarto atual é o "11" e o tempo acaba), simulando a lógica que será implementada no módulo principal.

Posteriormente, é utilizado um `PROCESS` responsável pelo `clock`, que alterna de '0' para '1' e vice-versa a cada 10 ns. O processo de estímulos (`tb_process`) inicia com o `reset` em '1' e, após 40 ns, passa para '0'. A partir desse momento, o código simula a progressão do jogo: altera o EA para iniciar a contagem, realiza pausas, aplica cargas manuais e aciona o `modo_novoquarto` para avançar os períodos. Esta rotina é repetida sistematicamente até ao quarto período (Q4), culminando num teste final do botão de novo quarto para certificar que a trava de `fim_jogo` funciona corretamente e impede novos avanços.

Segue abaixo o código desenvolvido, considerando que, no relatório, estão a ser apresentadas somente as partes funcionais do código, deixando de lado as bibliotecas utilizadas e a entidade vazia. Devido à sua extensão, o código foca na integração estrutural e nos processos de teste.

Architecture de `tb_cont.vhd`

```

8  ARCHITECTURE behavior OF tb_cont IS
9      SIGNAL clock: std_logic := '0';
10     SIGNAL reset: std_logic := '0';
11     SIGNAL fim_quarto: std_logic := '0';
12     SIGNAL fim_jogo: std_logic := '0';
13     SIGNAL EA: std_logic_vector(1 DOWNTO 0) := "00";
14     SIGNAL modo_novoquarto: std_logic := '0';
15     SIGNAL modo_nba: std_logic := '0';
16     SIGNAL passou_1cent: std_logic := '0';
17     SIGNAL passou_1seg: std_logic := '0';
18     SIGNAL passou_1min: std_logic := '0';
19     SIGNAL carga: std_logic := '0';
20     SIGNAL c_seg: std_logic_vector(1 DOWNTO 0) := std_logic_vector(
        to_unsigned(0, 2));
21     SIGNAL c_min: std_logic_vector(3 DOWNTO 0) := std_logic_vector(
        to_unsigned(0, 4));
22     SIGNAL c_quarto: std_logic_vector(1 DOWNTO 0) := std_logic_vector(

```

```

23         to_unsigned(0, 2));
24     SIGNAL centesimos: std_logic_vector(6 DOWNT0 0) := std_logic_vector(
        to_unsigned(0, 7));
25     SIGNAL segundos: std_logic_vector(5 DOWNT0 0) := std_logic_vector(
        to_unsigned(0, 6));
26     SIGNAL minutos: std_logic_vector(3 DOWNT0 0) := std_logic_vector(
        to_unsigned(0, 4));
27     SIGNAL quarto: std_logic_vector(1 DOWNT0 0) := std_logic_vector(
        to_unsigned(0, 2));
28 BEGIN
29     UUT1_divisor_clk: ENTITY work.divisor_clock
30         GENERIC MAP(
31             CLOCK_FREQ => 4
32         )
33         PORT MAP(
34             clock => clock,
35             reset => reset,
36             passou_1cent => passou_1cent
37         );
38
39     UUT2_cont_cent: ENTITY work.cont_cent
40         PORT MAP(
41             clock => clock,
42             reset => reset,
43             fim_quarto => fim_quarto,
44             EA => EA,
45             passou_1cent => passou_1cent,
46             modo_novoquarto => modo_novoquarto,
47             carga => carga,
48             centesimos => centesimos,
49             passou_1seg => passou_1seg,
50             fim_jogo => fim_jogo
51         );
52
53     UUT3_cont_seg: ENTITY work.cont_seg
54         PORT MAP(
55             clock => clock,
56             reset => reset,
57             fim_quarto => fim_quarto,
58             EA => EA,
59             passou_1seg => passou_1seg,
60             modo_novoquarto => modo_novoquarto,
61             carga => carga,
62             c_seg => c_seg,
63             segundos => segundos,
64             passou_1min => passou_1min,

```

```

65         fim_jogo => fim_jogo
66     );
67
68     UUT4_cont_min: ENTITY work.cont_min
69     PORT MAP(
70         clock => clock,
71         reset => reset,
72         EA => EA,
73         passou_1min => passou_1min,
74         modo_novoquarto => modo_novoquarto,
75         modo_nba => modo_nba,
76         carga => carga,
77         c_min => c_min,
78         minutos => minutos,
79         fim_quarto => fim_quarto,
80         fim_jogo => fim_jogo
81     );
82
83     UUT5_cont_quarto: ENTITY work.cont_quarto
84     PORT MAP(
85         clock => clock,
86         reset => reset,
87         EA => EA,
88         carga => carga,
89         modo_novoquarto => modo_novoquarto,
90         c_quarto => c_quarto,
91         quarto => quarto,
92         fim_jogo => fim_jogo
93     );
94
95     fim_quarto <= '1' WHEN (minutos = "0000" AND segundos = "000000" AND
96         centesimos = "0000000") ELSE '0';
97     fim_jogo <= '1' WHEN (quarto = "11" AND fim_quarto = '1') ELSE '0';
98
99     clock_process: PROCESS
100     BEGIN
101         clock <= '0';
102         WAIT FOR 10 ns;
103         clock <= '1';
104         WAIT FOR 10 ns;
105     END PROCESS;
106
107     tb_process: PROCESS
108     BEGIN
109         reset <= '1';
110         EA <= "00";
111         modo_nba <= '0';

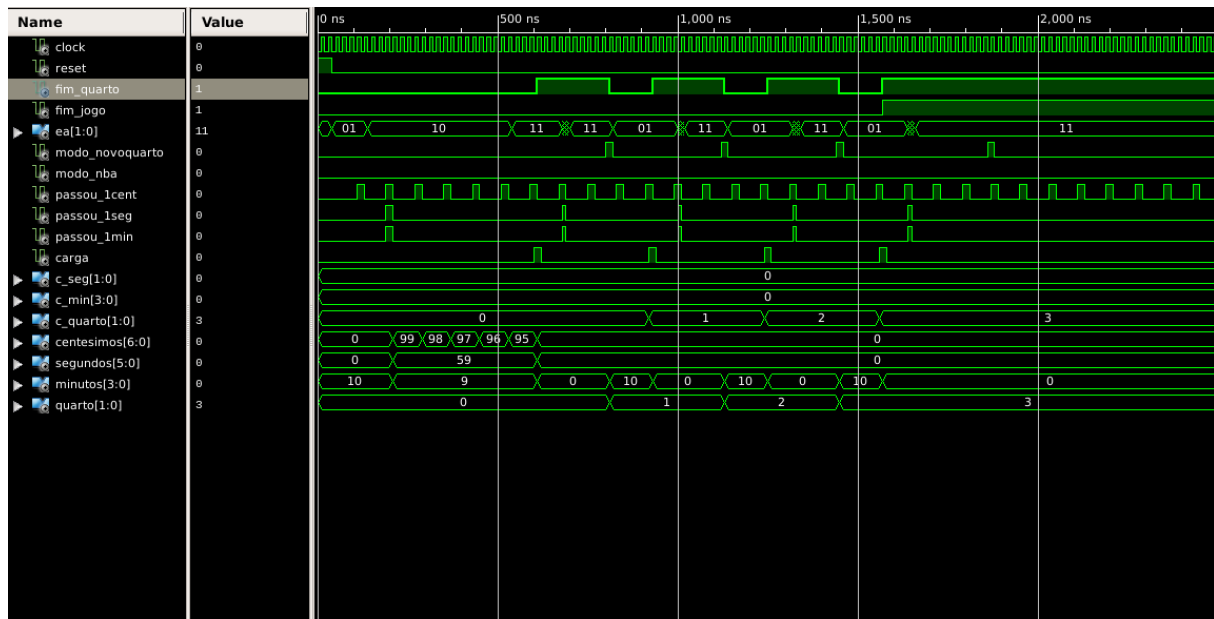
```

```
111     WAIT FOR 40 ns;
112
113     reset <= '0';
114     EA <= "01";
115     WAIT FOR 100 ns;
116
117     EA <= "10";
118     WAIT FOR 400 ns;
119
120     EA <= "11";
121     WAIT FOR 60 ns;
122
123     c_quarto <= "00";
124     c_min <= std_logic_vector(to_unsigned(0, 4));
125     c_seg <= "00";
126     carga <= '1';
127     WAIT FOR 20 ns;
128     carga <= '0';
129     WAIT FOR 60 ns;
130
131     EA <= "10";
132     WAIT FOR 20 ns;
133     EA <= "11";
134     WAIT FOR 100 ns;
135
136     modo_novoquarto <= '1';
137     WAIT FOR 20 ns;
138     modo_novoquarto <= '0';
139     EA <= "01";
140     WAIT FOR 100 ns;
141
142     c_quarto <= "01";
143     c_min <= std_logic_vector(to_unsigned(0, 4));
144     c_seg <= "00";
145     carga <= '1';
146     WAIT FOR 20 ns;
147     carga <= '0';
148     WAIT FOR 60 ns;
149
150     EA <= "10";
151     WAIT FOR 20 ns;
152     EA <= "11";
153     WAIT FOR 100 ns;
154
155     modo_novoquarto <= '1';
156     WAIT FOR 20 ns;
157     modo_novoquarto <= '0';
```

```
158     EA <= "01";
159     WAIT FOR 100 ns;
160
161     c_quarto <= "10";
162     c_min <= std_logic_vector(to_unsigned(0, 4));
163     c_seg <= "00";
164     carga <= '1';
165     WAIT FOR 20 ns;
166     carga <= '0';
167     WAIT FOR 60 ns;
168
169     EA <= "10";
170     WAIT FOR 20 ns;
171     EA <= "11";
172     WAIT FOR 100 ns;
173
174     modo_novoquarto <= '1';
175     WAIT FOR 20 ns;
176     modo_novoquarto <= '0';
177     EA <= "01";
178     WAIT FOR 100 ns;
179
180     c_quarto <= "11";
181     c_min <= std_logic_vector(to_unsigned(0, 4));
182     c_seg <= "00";
183     carga <= '1';
184     WAIT FOR 20 ns;
185     carga <= '0';
186     WAIT FOR 60 ns;
187
188     EA <= "10";
189     WAIT FOR 20 ns;
190     EA <= "11";
191     WAIT FOR 200 ns;
192
193     modo_novoquarto <= '1';
194     WAIT FOR 20 ns;
195     modo_novoquarto <= '0';
196     WAIT FOR 200 ns;
197
198     WAIT;
199 END PROCESS;
200
201 END behavior;
```

Abaixo é apresentada uma imagem referente às ondas geradas pelo *testbench* implementado. É possível notar que o funcionamento da cascata está correto: a cada virada do contador de centésimos, os segundos são decrementados, e o mesmo ocorre com os minutos. Observa-se também a progressão dos quartos controlada pelo botão de *novo quarto* e o bloqueio final de contagem assim que a partida é finalizada.

Figura 3 – Formas de onda da simulação dos contadores.



Fonte: Do autor (2026).

2.1.7 Máquina de Estados Finitos (FSM)

A Máquina de Estados Finitos (FSM) atua como o "cérebro" do cronômetro, sendo a responsável por coordenar todas as operações do circuito. É ela que avalia os comandos do usuário e o progresso do tempo para decidir se os contadores devem decrementar, aceitar uma carga manual ou simplesmente aguardar. O seu desenvolvimento baseou-se no diagrama de estados sugerido na especificação do projeto, adaptado para lidar com o fim de jogo e as transições de quarto.

2.1.7.1 Implementação

Abaixo é apresentada a entidade do módulo, que concentra os sinais de controle do sistema.

Entity de fsm.vhd

```

4 ENTITY fsm IS
5     PORT (
6         clock: IN std_logic;
```

```

7      reset: IN std_logic;
8      para_continua: IN std_logic;
9      modo_novoquarto: IN std_logic;
10     fim_jogo: IN std_logic;
11     fim_quarto: IN std_logic;
12     EA: OUT std_logic_vector(1 DOWNT0 0)
13 );
14 END entity;
```

Fonte: Do autor (2026)

As entradas da FSM misturam comandos externos gerados pelos botões da placa Nexys (*para_continua*, *modo_novoquarto*, *reset*) e sinais internos de aviso gerados pela própria lógica do circuito (*fim_quarto* e *fim_jogo*). A única saída do módulo é o vetor EA (Estado Atual) de 2 bits, que é distribuído para todos os contadores do *datapath* para ditar qual ação eles devem tomar em determinado momento de *clock*.

Na ARCHITECTURE, a estrutura de estados foi definida através da criação de um tipo enumerado (*states*), contendo quatro fases de operação: *MODE_SET*, *IDLE*, *COUNTING* e *STOPPED*. A escrita seguiu o modelo tradicional de máquinas de estados em VHDL, separando a lógica síncrona (memória) da lógica combinacional (transições). O processo síncrono é responsável por atualizar o estado atual na borda de subida do relógio e garantir que, durante um *reset* assíncrono, a máquina retorne ao estado inicial de configuração (*MODE_SET*).

Início da Architecture e Lógica Síncrona

```

16 ARCHITECTURE behavior OF fsm IS
17     TYPE states IS (MODE_SET, IDLE, COUNTING, STOPPED);
18     SIGNAL estado_atual, proximo_estado: states;
19 BEGIN
20     PROCESS(clock, reset)
21     BEGIN
22         IF reset = '1' THEN
23             estado_atual <= MODE_SET;
24         ELSIF rising_edge(clock) THEN
25             estado_atual <= proximo_estado;
26         END IF;
27     END PROCESS;
```

Fonte: Do autor (2026)

A parte central da inteligência do sistema reside no processo combinacional. Este bloco avalia o *estado_atual* e as entradas para definir qual será o *proximo_estado*. O estado *MODE_SET* funciona apenas como uma transição imediata para *IDLE*, permitindo que o registrador de modo do módulo de topo capture se o usuário selecionou as regras da FIBA ou NBA durante o *reset*.

No estado IDLE (espera), a contagem apenas inicia (indo para COUNTING) se o usuário pressionar para_continua e se as travas de segurança indicarem que o jogo ou o quarto não terminaram. Já no estado COUNTING (contagem ativa), o sistema pode ser pausado manualmente pelo mesmo botão, ou interrompido automaticamente caso o sinal fim_quarto seja ativado. Em ambos os casos, a máquina avança para o estado STOPPED.

Por fim, quando a máquina está em STOPPED (parada), existem dois caminhos possíveis. Se a partida não tiver acabado, o usuário pode pressionar para_continua para retomar o decréscimo do tempo, voltando para COUNTING. Alternativamente, se for o momento de iniciar um novo período, o comando modo_novoquarto desloca a máquina de volta para IDLE, preparando os contadores para um novo quarto de jogo.

Lógica Combinacional de Transição de Estados

```

29 PROCESS(estado_atual, para_continua, fim_quarto, modo_novoquarto,
    fim_jogo)
30 BEGIN
31     proximo_estado <= estado_atual;
32     CASE estado_atual IS
33         WHEN MODE_SET =>
34             proximo_estado <= IDLE;
35
36         WHEN IDLE =>
37             IF para_continua = '1' AND fim_quarto = '0' AND fim_jogo
               = '0' THEN
38                 proximo_estado <= COUNTING;
39             END IF;
40
41         WHEN COUNTING =>
42             IF para_continua = '1' OR fim_quarto = '1' THEN
43                 proximo_estado <= STOPPED;
44             END IF;
45
46         WHEN STOPPED =>
47             IF para_continua = '1' AND fim_quarto = '0' AND fim_jogo
               = '0' THEN
48                 proximo_estado <= COUNTING;
49             ELSIF modo_novoquarto = '1' AND fim_jogo = '0' THEN
50                 proximo_estado <= IDLE;
51             END IF;
52     END CASE;
53 END PROCESS;

```

Fonte: Do autor (2026)

Para finalizar o módulo, o tipo enumerado utilizado internamente (states) precisa ser traduzido para um formato binário compreensível pelos outros componentes de *hardware*. Fora

dos processos, uma estrutura combinacional `WITH SELECT` atribui um valor de 2 bits à porta de saída EA correspondente ao estado atual da máquina, fechando o ciclo de controle.

Codificação da Saída do Estado Atual

```

55     WITH estado_atual SELECT
56         EA <= "00" WHEN MODE_SET ,
57             "01" WHEN IDLE ,
58             "10" WHEN COUNTING ,
59             "11" WHEN STOPPED ,
60             "00" WHEN others;
61
62 END ARCHITECTURE;
```

Fonte: Do autor (2026)

2.1.7.2 Validação

Para a validação da Máquina de Estados Finitos, foi desenvolvido um *testbench* dedicado a simular as transições entre as fases operacionais do sistema. Nele, são gerados os estímulos de entrada para replicar as ações do utilizador e os avisos do bloco de dados, permitindo observar como o vetor de saída EA reage a cada combinação de eventos. Após a declaração dos sinais internos, é instanciado o submódulo `fsm.vhd` e efetuado o mapeamento das portas.

O processo de *clock* foi configurado para alternar de '0' para '1' a cada 10 ns, criando a base de tempo do circuito.

Architecture e Instanciação de `tb_fsm.vhd`

```

7  ARCHITECTURE behavior OF tb_fsm IS
8      SIGNAL clock: std_logic := '0';
9      SIGNAL reset: std_logic := '0';
10     SIGNAL para_continua: std_logic := '0';
11     SIGNAL modo_novoquarto: std_logic := '0';
12     SIGNAL fim_jogo: std_logic := '0';
13     SIGNAL fim_quarto: std_logic := '0';
14     SIGNAL EA: std_logic_vector(1 DOWNTO 0);
15
16 BEGIN
17     UUT: ENTITY work.fsm
18         PORT MAP (
19             clock => clock,
20             reset => reset,
21             para_continua => para_continua,
22             modo_novoquarto => modo_novoquarto,
23             fim_jogo => fim_jogo,
24             fim_quarto => fim_quarto,
25             EA => EA
```

```

26         );
27
28     clock_process: PROCESS
29 BEGIN
30     clock <= '0';
31     WAIT FOR 10 ns;
32     clock <= '1';
33     WAIT FOR 10 ns;
34 END PROCESS;

```

Fonte: Do autor (2026)

Posteriormente, é implementado o processo de estímulos (*estimulos_process*). Este bloco foi desenhado para testar exaustivamente todos os caminhos do diagrama de estados. O teste inicia com o acionamento do *reset* por 40 ns, o que força a máquina a passar pelo estado de configuração e repousar em IDLE ("01").

A partir desse momento, o código simula a pressão do botão *para_continua*, levando a máquina para o estado COUNTING ("10"). O teste prossegue realizando uma pausa e uma retoma manual, certificando que o sistema entra e sai do estado STOPPED ("11") corretamente de acordo com os comandos do utilizador. Na sequência, é injetado o aviso interno de *fim_quarto*, forçando a máquina a parar de contar de forma automática, seguido de um pulso em *modo_novoquarto* para fazer o sistema retornar ao estado IDLE ("01") e aguardar o início de um novo período.

Por fim, o código simula a conclusão da partida ao ativar as restrições *fim_quarto* e *fim_jogo* simultaneamente. Com o sistema bloqueado em STOPPED ("11"), o *testbench* emite novos pulsos nos botões *modo_novoquarto* e *para_continua* para provar que a trava de segurança funciona, impedindo que a máquina saia do estado final.

Processo de Estímulos de *tb_fsm.vhd*

```

36     estimulos_process: PROCESS
37 BEGIN
38     reset <= '1';
39     WAIT FOR 40 ns;
40     reset <= '0';
41     WAIT FOR 40 ns;
42
43     para_continua <= '1';
44     WAIT FOR 20 ns;
45     para_continua <= '0';
46     WAIT FOR 60 ns;
47
48     para_continua <= '1';
49     WAIT FOR 20 ns;
50     para_continua <= '0';

```

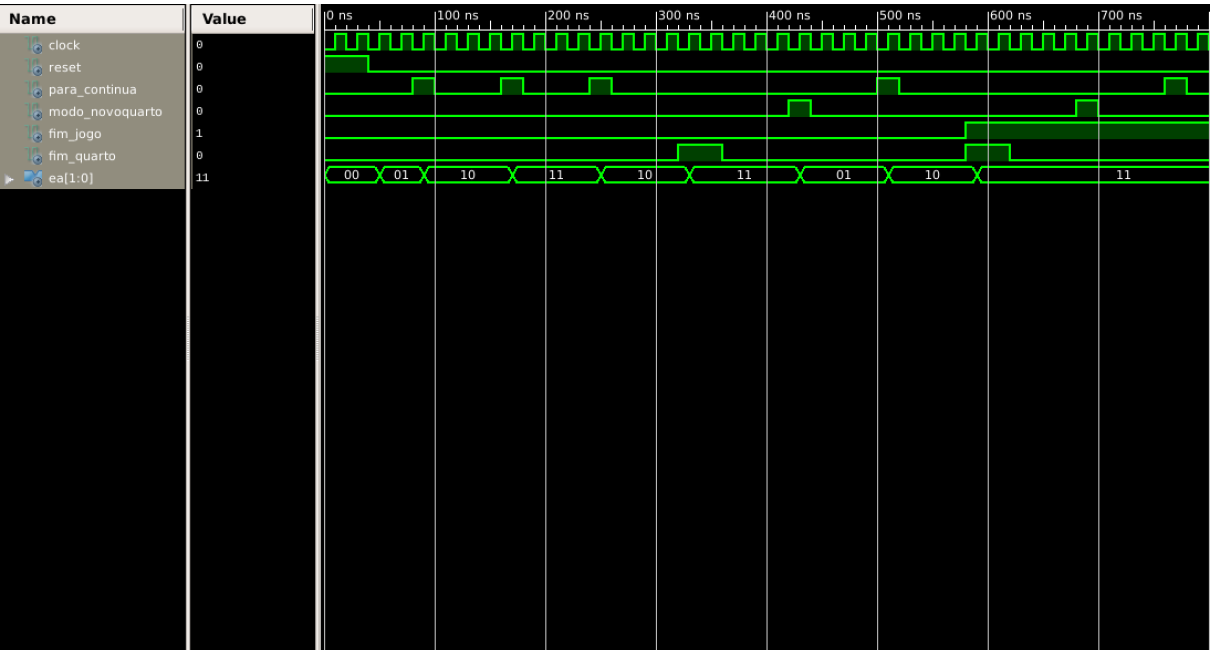
```
51     WAIT FOR 60 ns;
52
53     para_continua <= '1';
54     WAIT FOR 20 ns;
55     para_continua <= '0';
56     WAIT FOR 60 ns;
57
58     fim_quarto <= '1';
59     WAIT FOR 40 ns;
60     fim_quarto <= '0';
61     WAIT FOR 60 ns;
62
63     modo_novoquarto <= '1';
64     WAIT FOR 20 ns;
65     modo_novoquarto <= '0';
66     WAIT FOR 60 ns;
67
68     para_continua <= '1';
69     WAIT FOR 20 ns;
70     para_continua <= '0';
71     WAIT FOR 60 ns;
72
73     fim_quarto <= '1';
74     fim_jogo <= '1';
75     WAIT FOR 40 ns;
76     fim_quarto <= '0';
77     WAIT FOR 60 ns;
78
79     modo_novoquarto <= '1';
80     WAIT FOR 20 ns;
81     modo_novoquarto <= '0';
82     WAIT FOR 60 ns;
83
84     para_continua <= '1';
85     WAIT FOR 20 ns;
86     para_continua <= '0';
87     WAIT FOR 60 ns;
88
89     WAIT;
90     END PROCESS;
91
92 END behavior;
```

Fonte: Do autor (2026)

Abaixo é apresentada uma imagem referente às ondas geradas pelo simulador. Através dela, é possível confirmar o comportamento esperado da máquina de estados: as transições de

"01" para "10" com o *play*, as idas a "11" durante as pausas ou finais de período, o retorno a "01" com o novo quarto, e a estabilidade final imposta pela flag de término de jogo, validando a lógica de controle principal.

Figura 4 – Formas de onda da FSM.



Fonte: Do autor (2026).

2.1.8 Módulo independente de plataforma

O módulo independente de plataforma atua como o núcleo lógico principal do projeto. A sua função é reunir, instanciar e interligar todos os submódulos desenvolvidos anteriormente, formando o fluxo de dados completo do cronómetro. Este módulo recebe o sufixo "PI" (do inglês *Platform-Independent*) justamente por agrupar apenas a lógica pura do sistema, não possuindo dependência de componentes físicos específicos de uma placa comercial, como filtros mecânicos de botões ou controladores visuais.

2.1.8.1 Implementação

Abaixo é apresentada a entidade do módulo, que consolida todas as entradas de controle e saídas de tempo.

Entity de cron_basq_PI.vhd

```
5 ENTITY cron_basq_PI IS
6     GENERIC (
7         CLOCK_FREQ: integer := 500000
8     );
9     PORT (
```

```

10     clock: IN std_logic;
11     reset: IN std_logic;
12     para_continua: IN std_logic;
13     modo_novoquarto: IN std_logic;
14     carga: IN std_logic;
15     c_quarto: IN std_logic_vector(1 DOWNTO 0);
16     c_min: IN std_logic_vector(3 DOWNTO 0);
17     c_seg: IN std_logic_vector(1 DOWNTO 0);
18     quarto: OUT std_logic_vector(1 DOWNTO 0);
19     minutos: OUT std_logic_vector(3 DOWNTO 0);
20     segundos: OUT std_logic_vector(5 DOWNTO 0);
21     centesimos: OUT std_logic_vector(6 DOWNTO 0)
22 );
23 END ENTITY cron_basq_PI;

```

Fonte: Do autor (2026)

A entidade utiliza o parâmetro genérico `CLOCK_FREQ`, o que permite adaptar facilmente a velocidade do relógio para diferentes simulações ou plataformas físicas no futuro. As portas englobam todos os botões e chaves definidos nos contadores individuais, e as saídas entregam o estado atualizado do jogo para a camada superior.

Na `ARCHITECTURE`, foram declarados diversos sinais internos. Esses sinais são fundamentais para ler as saídas dos módulos instanciados e utilizá-las em outras partes do código simultaneamente. O primeiro processo síncrono do bloco é dedicado à configuração da regra do jogo. Durante o *reset*, o sistema avalia o estado do botão `modo_novoquarto`. Essa escolha é guardada no registrador `modo_nba`, servindo de aviso para o contador de minutos inicializar a partida com o padrão FIBA ou NBA.

Declaração de Sinais e Lógica de Modo

```

25 ARCHITECTURE behavior OF cron_basq_PI IS
26     SIGNAL fim_quarto: std_logic := '0';
27     SIGNAL fim_jogo: std_logic := '0';
28     SIGNAL EA: std_logic_vector(1 DOWNTO 0) := "00";
29     SIGNAL modo_nba: std_logic := '0';
30     SIGNAL passou_1cent: std_logic := '0';
31     SIGNAL passou_1seg: std_logic := '0';
32     SIGNAL passou_1min: std_logic := '0';
33
34     SIGNAL centesimos_int: std_logic_vector(6 DOWNTO 0);
35     SIGNAL segundos_int: std_logic_vector(5 DOWNTO 0);
36     SIGNAL minutos_int: std_logic_vector(3 DOWNTO 0);
37     SIGNAL quarto_int: std_logic_vector(1 DOWNTO 0);
38
39 BEGIN
40

```

```

41 process_mod0: PROCESS(clock, reset)
42 BEGIN
43     IF reset = '1' THEN
44         modo_nba <= modo_novoquarto;
45     END IF;
46 END PROCESS;

```

Fonte: Do autor (2026)

A etapa seguinte consiste no mapeamento estrutural (*Port Map*). Todos os contadores, o divisor de *clock* e a máquina de estados são instanciados dentro desta arquitetura. As ligações garantem que o estado atual gerado pela máquina (EA) chegue a todos os contadores de tempo e que a cascata flua em perfeita sincronia, ligando o aviso *passou_1cent* aos centésimos, o *passou_1seg* aos segundos e o *passou_1min* aos minutos. Para manter o relatório conciso, o trecho abaixo ilustra apenas a instanciación da FSM e do contador de minutos, demonstrando a interligação dos sinais de controle.

Exemplo de Instanciación Estrutural

```

48 inst_fsm: ENTITY work.fsm
49     PORT MAP(
50         clock => clock,
51         reset => reset,
52         para_continua => para_continua,
53         modo_novoquarto => modo_novoquarto,
54         fim_jogo => fim_jogo,
55         fim_quarto => fim_quarto,
56         EA => EA
57     );
58
59 inst_divisor_clk: ENTITY work.divisor_clock
60     GENERIC MAP(
61         CLOCK_FREQ => CLOCK_FREQ
62     )
63     PORT MAP(
64         clock => clock,
65         reset => reset,
66         passou_1cent => passou_1cent
67     );
68
69 inst_cont_cent: ENTITY work.cont_cent
70     PORT MAP(
71         clock => clock,
72         reset => reset,
73         fim_quarto => fim_quarto,
74         EA => EA,
75         passou_1cent => passou_1cent,

```

```

76         modo_novoquarto => modo_novoquarto,
77         carga => carga,
78         centesimos => centesimos_int,
79         passou_1seg => passou_1seg,
80         fim_jogo => fim_jogo
81     );
82
83     inst_cont_seg: ENTITY work.cont_seg
84     PORT MAP(
85         clock => clock,
86         reset => reset,
87         fim_quarto => fim_quarto,
88         EA => EA,
89         passou_1seg => passou_1seg,
90         modo_novoquarto => modo_novoquarto,
91         carga => carga,
92         c_seg => c_seg,
93         segundos => segundos_int,
94         passou_1min => passou_1min,
95         fim_jogo => fim_jogo
96     );
97
98     inst_cont_min: ENTITY work.cont_min
99     PORT MAP(
100         clock => clock,
101         reset => reset,
102         EA => EA,
103         passou_1min => passou_1min,
104         modo_novoquarto => modo_novoquarto,
105         modo_nba => modo_nba,
106         carga => carga,
107         c_min => c_min,
108         minutos => minutos_int,
109         fim_quarto => fim_quarto,
110         fim_jogo => fim_jogo
111     );
112
113     inst_cont_quarto: ENTITY work.cont_quarto
114     PORT MAP(
115         clock => clock,
116         reset => reset,
117         EA => EA,
118         carga => carga,
119         modo_novoquarto => modo_novoquarto,
120         c_quarto => c_quarto,
121         quarto => quarto_int,
122         fim_jogo => fim_jogo

```

123);

Fonte: Do autor (2026)

Por fim, a arquitetura contém atribuições combinacionais vitais para a autonomia do cronômetro. Como os submódulos não sabem quando a partida inteira acaba, essa inteligência foi centralizada no núcleo. O sinal `fim_quarto` é ativado no exato instante em que os vetores de minutos, segundos e centésimos chegam a zero de forma simultânea. Utilizando esta conclusão, o sinal `fim_jogo` é acionado caso o sistema esteja no quarto período ("11"). Estas travas lógicas são então distribuídas de volta aos módulos estruturais, fechando a malha de controle antes de direcionar os valores finais para as portas de saída.

Geração de Travas e Saídas Combinacionais

```

125     fim_quarto <= '1' WHEN (minutos_int = "0000" AND segundos_int = "
126         000000" AND centesimos_int = "0000000") ELSE '0';
127
128     fim_jogo <= '1' WHEN (quarto_int = "11" AND fim_quarto = '1') ELSE
129         '0';
130
131     quarto <= quarto_int;
132     minutos <= minutos_int;
133     segundos <= segundos_int;
134     centesimos <= centesimos_int;
135
136 END behavior;
```

Fonte: Do autor (2026)

2.1.8.2 Validação

Para a validação do módulo independente de plataforma, foi desenvolvido um *testbench* abrangente, projetado para simular uma partida inteira em escala de tempo reduzida. O objetivo principal deste teste é comprovar que o núcleo lógico do sistema consegue coordenar adequadamente a Máquina de Estados, o divisor de *clock* e todos os contadores em conjunto, respondendo corretamente aos estímulos externos.

Após a declaração dos sinais, o submódulo `cron_basq_PI` é instanciado. Um detalhe crucial nesta etapa é a reconfiguração do parâmetro `CLOCK_FREQ` para o valor 2. Na implementação física, este valor é 500.000 (para gerar centésimos a partir de 50 MHz), mas reduzi-lo no ambiente de simulação permite visualizar o decréscimo de tempo nas formas de onda sem a necessidade de processar milhões de ciclos de relógio, facilitando a análise visual.

```

8  ARCHITECTURE behavior OF tb_cron_basq_PI IS
9      SIGNAL clock: std_logic := '0';
10     SIGNAL reset: std_logic := '0';
11     SIGNAL para_continua: std_logic := '0';
12     SIGNAL modo_novoquarto: std_logic := '0';
13     SIGNAL carga: std_logic := '0';
14     SIGNAL c_quarto: std_logic_vector(1 DOWNTO 0) := "00";
15     SIGNAL c_min: std_logic_vector(3 DOWNTO 0) := "0000";
16     SIGNAL c_seg: std_logic_vector(1 DOWNTO 0) := "00";
17     SIGNAL quarto: std_logic_vector(1 DOWNTO 0);
18     SIGNAL minutos: std_logic_vector(3 DOWNTO 0);
19     SIGNAL segundos: std_logic_vector(5 DOWNTO 0);
20     SIGNAL centesimos: std_logic_vector(6 DOWNTO 0);
21
22 BEGIN
23     UUT: ENTITY work.cron_basq_PI
24         GENERIC MAP(
25             CLOCK_FREQ => 2
26         )
27         PORT MAP(
28             clock => clock,
29             reset => reset,
30             para_continua => para_continua,
31             modo_novoquarto => modo_novoquarto,
32             carga => carga,
33             c_quarto => c_quarto,
34             c_min => c_min,
35             c_seg => c_seg,
36             quarto => quarto,
37             minutos => minutos,
38             segundos => segundos,
39             centesimos => centesimos
40         );
41
42     clock_process: PROCESS
43 BEGIN
44     clock <= '0';
45     WAIT FOR 10 ns;
46     clock <= '1';
47     WAIT FOR 10 ns;
48 END PROCESS;

```

Fonte: Do autor (2026)

A rotina de estímulos (tb_process) foi estruturada para testar as transições de tempo e de estado de forma acelerada. O processo inicia com o reset ativo e o botão modo_novoquarto em

'0', o que configura a partida para o padrão FIBA (10 minutos) no primeiro quarto. Em seguida, o botão `para_continua` é pressionado duas vezes, testando o início e a pausa da contagem.

Com o cronómetro pausado, é injetada uma carga manual para "avançar no tempo", forçando o painel para o tempo de 0 minutos e 0 segundos do Q1 ("00"). Ao retomar a contagem, o tempo esgota rapidamente, ativando internamente a trava de `fim_quarto`. Logo após, o botão `modo_novoquarto` é simulado, confirmando que o sistema transita corretamente para o segundo quarto (Q2).

Para testar a conclusão da partida, uma nova carga manual é aplicada, saltando o jogo diretamente para o final do quarto período (Q4, "11") com tempo zerado. Ao prosseguir a contagem, o `fim_jogo` é alcançado. Neste estado de bloqueio absoluto, o código emite novos pulsos nos botões `carga` e `modo_novoquarto` para provar que a trava impede qualquer modificação indevida após o término do jogo. Por fim, um novo `reset` é acionado juntamente com o botão `modo_novoquarto` ativado, testando a comutação bem-sucedida para o modo de regras da NBA (12 minutos).

Processo de Estímulos de `tb_cron_basq_PI.vhd`

```

50  tb_process: PROCESS
51  BEGIN
52      modo_novoquarto <= '0';
53      reset <= '1';
54      WAIT FOR 40 ns;
55      reset <= '0';
56      WAIT FOR 100 ns;
57
58      para_continua <= '1';
59      WAIT FOR 20 ns;
60      para_continua <= '0';
61      WAIT FOR 200 ns;
62
63      para_continua <= '1';
64      WAIT FOR 20 ns;
65      para_continua <= '0';
66      WAIT FOR 100 ns;
67
68      c_quarto <= "00";
69      c_min <= std_logic_vector(to_unsigned(0, 4));
70      c_seg <= "00";
71      carga <= '1';
72      WAIT FOR 20 ns;
73      carga <= '0';
74      WAIT FOR 60 ns;
75
76      para_continua <= '1';
77      WAIT FOR 20 ns;
78      para_continua <= '0';

```

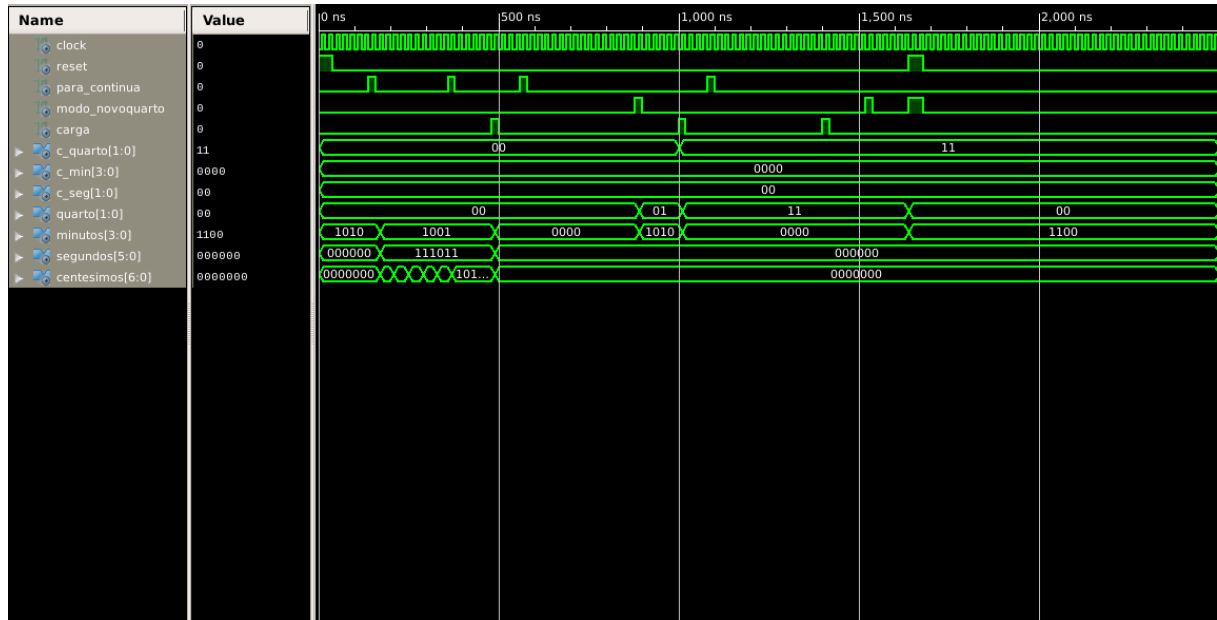
```
79      WAIT FOR 300 ns;
80
81      modo_novoquarto <= '1';
82      WAIT FOR 20 ns;
83      modo_novoquarto <= '0';
84      WAIT FOR 100 ns;
85
86      c_quarto <= "11";
87      c_min <= std_logic_vector(to_unsigned(0, 4));
88      c_seg <= "00";
89      carga <= '1';
90      WAIT FOR 20 ns;
91      carga <= '0';
92      WAIT FOR 60 ns;
93
94      para_continua <= '1';
95      WAIT FOR 20 ns;
96      para_continua <= '0';
97      WAIT FOR 300 ns;
98
99      carga <= '1';
100     WAIT FOR 20 ns;
101     carga <= '0';
102     WAIT FOR 100 ns;
103
104     modo_novoquarto <= '1';
105     WAIT FOR 20 ns;
106     modo_novoquarto <= '0';
107     WAIT FOR 100 ns;
108
109     modo_novoquarto <= '1';
110     reset <= '1';
111     WAIT FOR 40 ns;
112     reset <= '0';
113     modo_novoquarto <= '0';
114     WAIT FOR 100 ns;
115
116     WAIT;
117 END PROCESS;
118
119 END behavior;
```

Fonte: Do autor (2026)

Abaixo é apresentada uma imagem referente às ondas geradas pela simulação do núcleo independente. Através desta visualização, é possível atestar o correto funcionamento das funcio-

nalidades globais do sistema, como o controle de modo FIBA/NBA, a transição entre os quartos após as cargas de aceleração temporal e o acionamento das travas de término da partida.

Figura 5 – Formas de onda do módulo independente de plataforma.



Fonte: Do autor (2026).

2.1.9 Módulo de topo (Top-Level)

O módulo de topo é a camada final da hierarquia do projeto. A sua principal função é atuar como uma ponte entre a lógica independente de plataforma (o núcleo do cronômetro) e os componentes físicos da placa de desenvolvimento Nexys. É neste arquivo que ocorre o tratamento das limitações do *hardware* real, como o ruído mecânico dos botões, e a conversão dos dados binários para formatos compreensíveis pelos periféricos de saída visual, como os mostradores de sete segmentos e os LEDs.

2.1.9.1 Implementação

Abaixo apresenta-se a entidade do módulo, cujas portas representam as ligações diretas com os pinos físicos do FPGA.

Entity de top_cron_basq.vhd

```

5 ENTITY top_cron_basq IS
6     PORT (
7         clock: IN std_logic;
8         reset: IN std_logic;
9         carga: IN std_logic;
10        para_continua: IN std_logic;

```

```

11      modo_novoquarto: IN std_logic;
12      c_quarto: IN std_logic_vector(1 DOWNT0 0);
13      c_min: IN std_logic_vector(3 DOWNT0 0);
14      c_seg: IN std_logic_vector(1 DOWNT0 0);
15      led_quarto: OUT std_logic_vector(3 DOWNT0 0);
16      led_minutos: OUT std_logic_vector(3 DOWNT0 0);
17      an: OUT std_logic_vector(3 DOWNT0 0);
18      dec_ddp: OUT std_logic_vector(7 DOWNT0 0)
19  );
20  END ENTITY top_cron_basq;

```

Fonte: Do autor (2026)

As entradas recebem os estímulos do relógio principal da placa de 50 MHz, dos quatro botões e das oito chaves deslizantes. As saídas foram divididas para controlar os oitos LEDs (quatro para os quartos e quatro para os minutos) e o conjunto de *displays* multiplexados, onde an controla os ânodos (selecionando qual dígito acende) e dec_ddp controla os catodos (formando o desenho dos números).

Na ARCHITECTURE, a primeira estrutura declarada é uma memória ROM interna. Como o núcleo do cronômetro realiza contagens matemáticas em formato binário puro, é necessário converter esses valores para o formato BCD (*Binary-Coded Decimal*) antes de enviá-los ao controlador de vídeo. A matriz conv_to_BCD atua como uma tabela de consulta rápida: o valor binário entra como endereço e a ROM devolve o valor correspondente separado em dezenas e unidades lógicas. Também são declarados os sinais que serão utilizados.

Declaração da ROM de Conversão BCD

```

22  ARCHITECTURE behavior OF top_cron_basq IS
23      TYPE ROM IS ARRAY (0 TO 127) OF std_logic_vector(7 DOWNT0 0);
24      CONSTANT conv_to_BCD: ROM := (
25          "00000000", "00000001", "00000010", "00000011", "00000100",
26          "00000101", "00000110", "00000111", "00001000", "00001001",
27          "00010000", "00010001", "00010010", "00010011", "00010100",
28          "00010101", "00010110", "00010111", "00011000", "00011001",
29          "00100000", "00100001", "00100010", "00100011", "00100100",
30          "00100101", "00100110", "00100111", "00101000", "00101001",
31          "00110000", "00110001", "00110010", "00110011", "00110100",
32          "00110101", "00110110", "00110111", "00111000", "00111001",
33          "01000000", "01000001", "01000010", "01000011", "01000100",
34          "01000101", "01000110", "01000111", "01001000", "01001001",
35          "01010000", "01010001", "01010010", "01010011", "01010100",
36          "01010101", "01010110", "01010111", "01011000", "01011001",
37          "01100000", "01100001", "01100010", "01100011", "01100100",
38          "01100101", "01100110", "01100111", "01101000", "01101001",
39          "01110000", "01110001", "01110010", "01110011", "01110100",
40          "01110101", "01110110", "01110111", "01111000", "01111001",

```

```

41      "10000000", "10000001", "10000010", "10000011", "10000100",
42      "10000101", "10000110", "10000111", "10001000", "10001001",
43      "10010000", "10010001", "10010010", "10010011", "10010100",
44      "10010101", "10010110", "10010111", "10011000", "10011001",
45      "00000000", "00000000", "00000000", "00000000", "00000000",
46      "00000000", "00000000", "00000000", "00000000", "00000000",
47      "00000000", "00000000", "00000000", "00000000", "00000000",
48      "00000000", "00000000", "00000000", "00000000", "00000000",
49      "00000000", "00000000", "00000000", "00000000", "00000000",
50      "00000000", "00000000", "00000000"
51  );
52
53  SIGNAL deb_carga: std_logic;
54  SIGNAL deb_para_continua: std_logic;
55  SIGNAL deb_mod0_novoquarto: std_logic;
56
57  SIGNAL quarto_out: std_logic_vector(1 DOWNTO 0);
58  SIGNAL minutos_out: std_logic_vector(3 DOWNTO 0);
59  SIGNAL segundos_out: std_logic_vector(5 DOWNTO 0);
60  SIGNAL centesimos_out: std_logic_vector(6 DOWNTO 0);
61
62  SIGNAL bcd_seg: std_logic_vector(7 DOWNTO 0);
63  SIGNAL bcd_cent: std_logic_vector(7 DOWNTO 0);
64
65  SIGNAL d0_int: std_logic_vector(5 DOWNTO 0);
66  SIGNAL d1_int: std_logic_vector(5 DOWNTO 0);
67  SIGNAL d2_int: std_logic_vector(5 DOWNTO 0);
68  SIGNAL d3_int: std_logic_vector(5 DOWNTO 0);

```

Fonte: Do autor (2026)

O próximo passo é o mapeamento estrutural dos componentes. Antes de entregar os sinais dos botões ao núcleo do cronômetro, eles passam por instâncias do submódulo *Debounce*. Este filtro é obrigatório na implementação em *hardware*, pois os botões físicos geram múltiplas oscilações elétricas muito rápidas ao serem pressionados. Sem o *debounce*, a máquina de estados interpretaria um único toque do usuário como dezenas de pressões, causando comportamentos imprevisíveis. Após a filtragem, os sinais limpos (*deb_carga*, etc.) alimentam o módulo principal *cron_basq_PI*. Também é instanciado o *dsp1_drv*, o controlador responsável por varrer os *displays* em alta frequência para criar a ilusão de ótica de que todos estão acesos ao mesmo tempo.

Instanciação dos Filtros e Módulos Principais

```

70 BEGIN
71
72     inst_deb_carga: ENTITY work.Debounce
73         PORT MAP(
74             clock => clock,

```

```

75         reset => reset,
76         key => carga,
77         debkey => deb_carga
78     );
79
80     inst_deb_para_continua: ENTITY work.Debounce
81     PORT MAP(
82         clock => clock,
83         reset => reset,
84         key => para_continua,
85         debkey => deb_para_continua
86     );
87
88     inst_deb_modonovoquarto: ENTITY work.Debounce
89     PORT MAP(
90         clock => clock,
91         reset => reset,
92         key => modonovoquarto,
93         debkey => deb_modonovoquarto
94     );
95
96     inst_cron_pi: ENTITY work.cron_basq_PI
97     PORT MAP(
98         clock => clock,
99         reset => reset,
100         para_continua => deb_para_continua,
101         modonovoquarto => deb_modonovoquarto,
102         carga => deb_carga,
103         c_quarto => c_quarto,
104         c_min => c_min,
105         c_seg => c_seg,
106         quarto => quarto_out,
107         minutos => minutos_out,
108         segundos => segundos_out,
109         centesimos => centesimos_out
110     );
111
112     inst_dspl_drv: ENTITY work.dspl_drv
113     PORT MAP(
114         clock => clock,
115         reset => reset,
116         d3 => d3_int,
117         d2 => d2_int,
118         d1 => d1_int,
119         d0 => d0_int,
120         an => an,
121         dec_ddp => dec_ddp

```

122);

Fonte: Do autor (2026)

Por fim, a arquitetura elabora a formatação visual. Os valores binários dos segundos e centésimos são convertidos para vetores BCD de 8 bits usando a ROM. Em seguida, estes vetores são fatiados em blocos de 4 bits (representando cada dígito decimal individualmente) e agrupados com bits de controle, formando os vetores `d0_int` a `d3_int` lidos pelo *driver* do *display*. O dígito `d2_int` recebe o bit '0' em sua extremidade, comando que serve para ligar fisicamente o ponto decimal (*dot*) do mostrador, criando uma separação visual elegante entre os segundos e os centésimos.

Para a interface dos LEDs, o vetor dos minutos é encaminhado diretamente, visto que a especificação dita a sua exibição em binário puro. Já para o vetor dos quartos, utiliza-se a técnica de codificação *one-hot* através de uma estrutura `WITH SELECT`: dependendo do período atual (de "00" a "11"), apenas um único LED correspondente acende no painel, garantindo uma leitura intuitiva por parte do utilizador.

Conversão BCD e Controle de Saídas Visuais

```

124     bcd_seg <= conv_to_BCD(to_integer(unsigned(segundos_out)));
125     bcd_cent <= conv_to_BCD(to_integer(unsigned(centesimos_out)));
126
127     d3_int <= '1' & bcd_seg(7 DOWNTO 4) & '1';
128     d2_int <= '1' & bcd_seg(3 DOWNTO 0) & '0';
129     d1_int <= '1' & bcd_cent(7 DOWNTO 4) & '1';
130     d0_int <= '1' & bcd_cent(3 DOWNTO 0) & '1';
131
132     WITH quarto_out SELECT
133         led_quarto <= "0001" WHEN "00",
134                      "0010" WHEN "01",
135                      "0100" WHEN "10",
136                      "1000" WHEN "11",
137                      "0000" WHEN others;
138
139     led_minutos <= minutos_out;
140
141 END behavior;
```

Fonte: Do autor (2026)

2.1.9.2 Validação

Para a validação do módulo de topo, foi desenvolvido um *testbench* com o intuito de certificar a correta integração entre o núcleo lógico do cronômetro e os componentes de inter-

face física. Uma particularidade fundamental deste teste em relação aos anteriores é a presença dos filtros mecânicos (Debounce) instanciados nesta camada. Devido a isso, os tempos de acionamento dos botões no simulador precisaram ser ajustados. Enquanto nos testes das camadas inferiores um pulso de 20 ns era suficiente para acionar a Máquina de Estados, neste *testbench* os botões precisam ser mantidos em nível lógico alto por pelo menos 100 ns; caso contrário, o filtro interpretará o comando como ruído e o ignorará.

Após a declaração dos sinais, o submódulo `top_cron_basq` é instanciado. O relógio é mantido com o mesmo período de 20 ns (alternando a cada 10 ns), simulando fielmente o oscilador de 50 MHz da placa Nexys.

Architecture e Instanciação de `tb_top_cron_basq.vhd`

```

8  ARCHITECTURE behavior OF tb_top_cron_basq IS
9      SIGNAL clock: std_logic := '0';
10     SIGNAL reset: std_logic := '0';
11     SIGNAL carga: std_logic := '0';
12     SIGNAL para_continua: std_logic := '0';
13     SIGNAL modo_novoquarto: std_logic := '0';
14     SIGNAL c_quarto: std_logic_vector(1 DOWNTO 0) := "00";
15     SIGNAL c_min: std_logic_vector(3 DOWNTO 0) := "0000";
16     SIGNAL c_seg: std_logic_vector(1 DOWNTO 0) := "00";
17
18     SIGNAL led_quarto: std_logic_vector(3 DOWNTO 0);
19     SIGNAL an: std_logic_vector(3 DOWNTO 0);
20     SIGNAL dec_ddp: std_logic_vector(7 DOWNTO 0);
21
22 BEGIN
23     UUT: ENTITY work.top_cron_basq
24         PORT MAP(
25             clock => clock,
26             reset => reset,
27             carga => carga,
28             para_continua => para_continua,
29             modo_novoquarto => modo_novoquarto,
30             c_quarto => c_quarto,
31             c_min => c_min,
32             c_seg => c_seg,
33             led_quarto => led_quarto,
34             an => an,
35             dec_ddp => dec_ddp
36         );
37
38     clock_process: PROCESS
39 BEGIN
40     clock <= '0';
41     WAIT FOR 10 ns;
42     clock <= '1';

```

```

43         WAIT FOR 10 ns;
44     END PROCESS;

```

Fonte: Do autor (2026)

A rotina de estímulos (tb_process) testa as funcionalidades gerais do projeto a partir das portas externas. O sistema é inicialmente resetado, estabelecendo as condições iniciais. Em seguida, um pulso de 100 ns é enviado ao comando para_continua para iniciar a contagem, seguido de outro pulso para pausar o cronômetro.

Com o sistema parado, o código simula a inserção de uma carga manual completa: as chaves são configuradas para o segundo quarto ("01"), 5 minutos (5) e 30 segundos ("10"), e o botão carga é mantido alto pelo tempo necessário para atravessar o *debounce*. Após a carga, a contagem é retomada e pausada novamente. O teste prossegue emitindo um comando de modo_novoquarto para verificar o avanço de período e, por fim, aciona o reset durante o funcionamento para garantir que o sistema retorna ao estado de segurança com sucesso.

Processo de Estímulos de tb_top_cron_basq.vhd

```

46     tb_process: PROCESS
47     BEGIN
48         reset <= '1';
49         modo_novoquarto <= '0';
50         WAIT FOR 100 ns;
51         reset <= '0';
52         WAIT FOR 200 ns;
53
54         para_continua <= '1';
55         WAIT FOR 100 ns;
56         para_continua <= '0';
57         WAIT FOR 500 ns;
58
59         para_continua <= '1';
60         WAIT FOR 100 ns;
61         para_continua <= '0';
62         WAIT FOR 200 ns;
63
64         c_quarto <= "01";
65         c_min <= std_logic_vector(to_unsigned(5, 4));
66         c_seg <= "10";
67         carga <= '1';
68         WAIT FOR 100 ns;
69         carga <= '0';
70         WAIT FOR 200 ns;
71
72         para_continua <= '1';
73         WAIT FOR 100 ns;

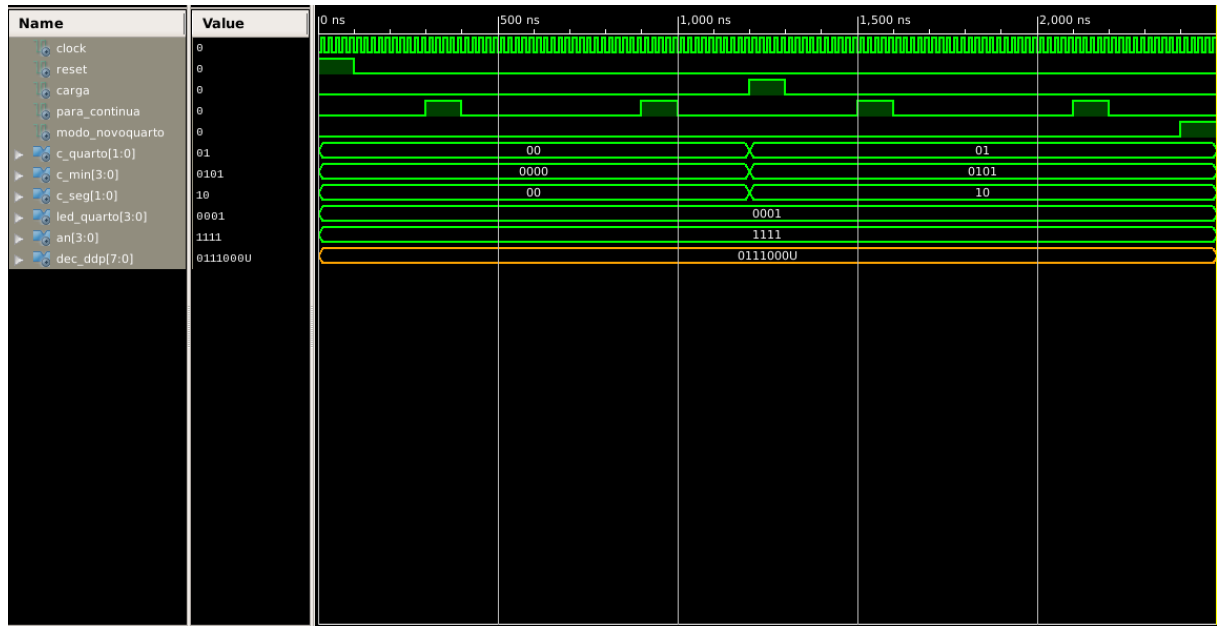
```

```
74     para_continua <= '0';
75     WAIT FOR 500 ns;
76
77     para_continua <= '1';
78     WAIT FOR 100 ns;
79     para_continua <= '0';
80     WAIT FOR 200 ns;
81
82     modo_novoquarto <= '1';
83     WAIT FOR 100 ns;
84     modo_novoquarto <= '0';
85     WAIT FOR 300 ns;
86
87     reset <= '1';
88     WAIT FOR 100 ns;
89     reset <= '0';
90     WAIT FOR 200 ns;
91
92     WAIT;
93 END PROCESS;
94
95 END behavior;
```

Fonte: Do autor (2026)

Abaixo é apresentada uma imagem referente às formas de onda geradas por este último *testbench*. Através dela, é possível observar o atraso intencional provocado pelos módulos de filtro nas entradas de botão e atestar a correta transição da saída `led_quarto` para a formatação *one-hot* definida na codificação visual, comprovando que o projeto está validado e pronto para a síntese na placa física. Vale ressaltar que o display não foi utilizado no *testbench*, por isso está não inicializado.

Figura 6 – Formas de onda do módulo de topo e interfaces.



Fonte: Do autor (2026).

2.2 PROTOTIPAÇÃO NA PLACA NEXYS 1

Para que o código desenvolvido e simulado em ambiente virtual pudesse ser testado fisicamente, foi realizada a prototipação do sistema na placa FPGA Nexys 1. O passo mais importante para essa integração física é a criação do arquivo de restrições do usuário, conhecido como UCF (*User Constraint File*). O objetivo deste arquivo é mapear as portas de entrada e saída, que foram definidas na entidade de topo do projeto, para os pinos físicos exatos onde os componentes estão soldados na placa.

2.2.1 Implementação do mapeamento

Abaixo é apresentado o código de mapeamento dos pinos no arquivo UCF, desenvolvido com base no esquemático elétrico da plataforma.

Mapeamento de pinos no arquivo .ucf

```

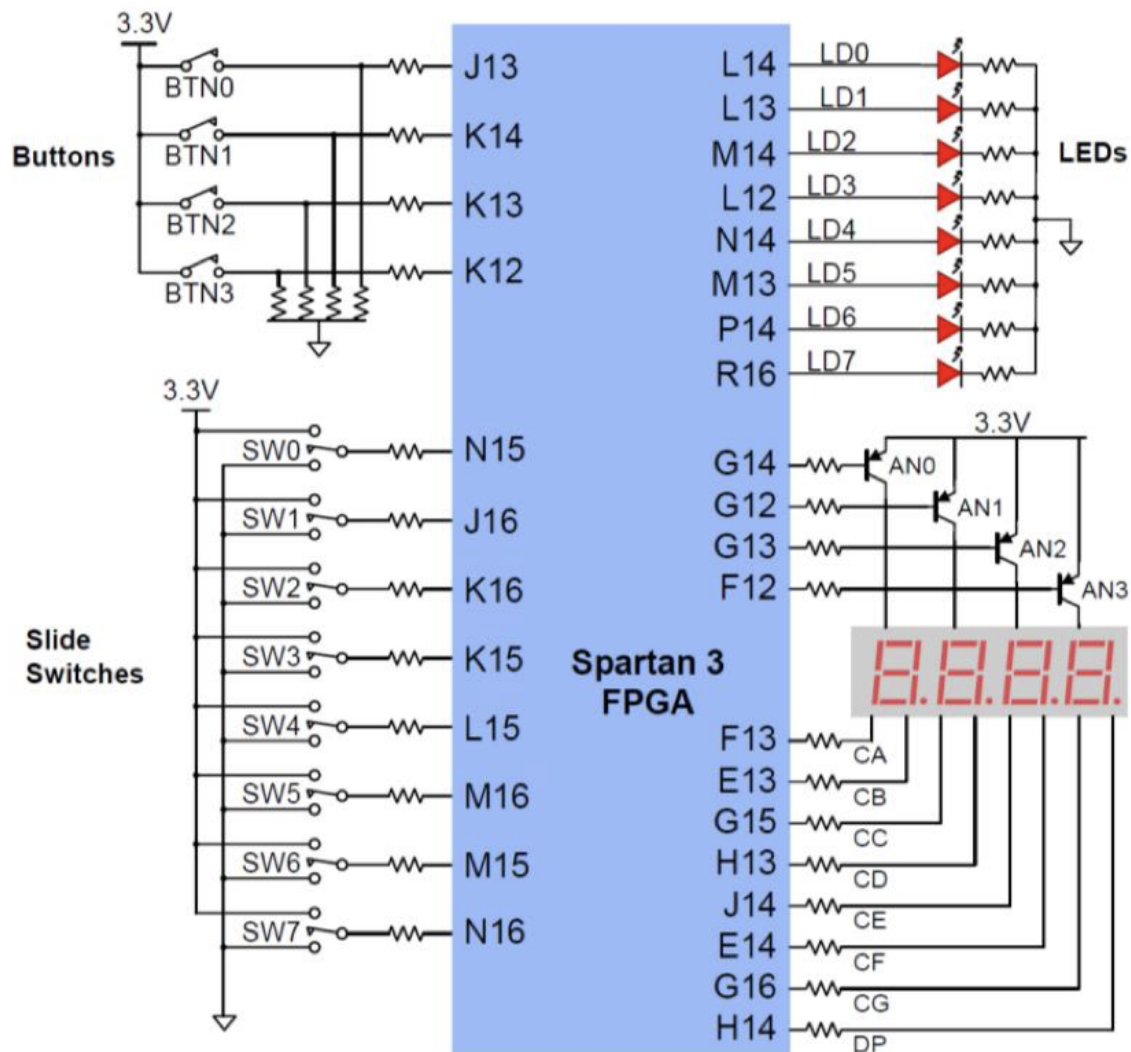
1 NET "clock" LOC = "T9";
2
3 NET "reset" LOC = "J13";
4 NET "reset" CLOCK_DEDICATED_ROUTE = FALSE;
5 NET "modo_novoquarto" LOC = "K14";
6 NET "para_continua" LOC = "K13";
7 NET "carga" LOC = "K12";
8
9 NET "c_quarto[1]" LOC = "N16";

```

```
10 NET "c_quarto[0]" LOC = "M15";
11
12 NET "c_min[3]" LOC = "M16";
13 NET "c_min[2]" LOC = "L15";
14 NET "c_min[1]" LOC = "K15";
15 NET "c_min[0]" LOC = "K16";
16
17 NET "c_seg[1]" LOC = "J16";
18 NET "c_seg[0]" LOC = "N15";
19
20 NET "led_quarto[3]" LOC = "R16";
21 NET "led_quarto[2]" LOC = "P14";
22 NET "led_quarto[1]" LOC = "M13";
23 NET "led_quarto[0]" LOC = "N14";
24
25 NET "led_minutos[3]" LOC = "L12";
26 NET "led_minutos[2]" LOC = "M14";
27 NET "led_minutos[1]" LOC = "L13";
28 NET "led_minutos[0]" LOC = "L14";
29
30 NET "an[3]" LOC = "F12";
31 NET "an[2]" LOC = "G13";
32 NET "an[1]" LOC = "G12";
33 NET "an[0]" LOC = "G14";
34
35 NET "dec_ddp[7]" LOC = "F13";
36 NET "dec_ddp[6]" LOC = "E13";
37 NET "dec_ddp[5]" LOC = "G15";
38 NET "dec_ddp[4]" LOC = "H13";
39 NET "dec_ddp[3]" LOC = "J14";
40 NET "dec_ddp[2]" LOC = "E14";
41 NET "dec_ddp[1]" LOC = "G16";
42 NET "dec_ddp[0]" LOC = "H14";
```

Fonte: Do autor (2026)

Figura 7 – Diagrama de pinos de periféricos da Nexys 1.



Fonte: CALAZANS, N. L. V. Material didático da disciplina Linguagem de Descrição de Hardware.

Como pode ser observado na listagem, a distribuição dos sinais seguiu rigorosamente a especificação do projeto e a arquitetura de *hardware* da Nexys 1. O sinal principal de `clock` foi conectado ao pino "T9", que provê a frequência base de 50 MHz através do oscilador de cristal da placa.

As entradas de controle do sistema foram associadas aos quatro botões de pressão (*push-buttons*). O `reset` foi mapeado para o botão BTN0 ("J13"). Um detalhe técnico importante nesta etapa foi a inclusão da restrição `CLOCK_DEDICATED_ROUTE = FALSE` logo após o mapeamento deste pino. Como o sinal de `reset` é distribuído para praticamente todos os módulos do projeto (possuindo um alto *fan-out*), o software de síntese tenta alocá-lo automaticamente em uma rede global e veloz dedicada a *clocks*. Contudo, como o pino do botão físico não é uma entrada construída para receber *clocks*, isso gerava um erro de mapeamento. A restrição adicionada resolve

esse conflito, instruindo o compilador a aceitar o roteamento padrão da placa, o que atende perfeitamente à leitura de um botão acionado manualmente. Os demais botões, `modo_novoquarto`, `para_continua` e `carga`, foram atribuídos a `BTN1` a `BTN3`, respetivamente.

Para a inserção de dados, o vetor de chaves deslizantes (*slide switches*) foi dividido em três blocos lógicos, partindo da esquerda para a direita: as duas primeiras chaves (`SW7` e `SW6`) designadas para os quartos (`c_quarto`), as quatro centrais (`SW5` a `SW2`) para os minutos (`c_min`), e as duas últimas (`SW1` e `SW0`) para o carregamento fracionado dos segundos (`c_seg`).

Para as saídas visuais, a placa possui oito LEDs que também foram seccionados em dois grupos. Os LEDs mais à esquerda (`LD7` a `LD4`) receberam as ligações do vetor `led_quarto` para indicar o período do jogo em codificação *one-hot*, enquanto os LEDs da direita (`LD3` a `LD0`) foram conectados a `led_minutos` para exibir o tempo restante em binário.

Por fim, ocorreu o roteamento do mostrador de sete segmentos multiplexado. Os quatro pinos de ânodo (`an`), responsáveis por habilitar cada um dos dígitos do visor, foram ligados às saídas de `AN3` a `AN0` ("F12" a "G14"). Os oito pinos de cátodo (`dec_ddp`), que moldam o desenho dos números ao acender segmentos específicos, foram interligados de "F13" até o "H14", incluindo o segmento `DP` que compõe o ponto decimal fixo entre os minutos e os centésimos.

2.2.2 Vídeio de demonstração

Segue abaixo um link do YouTube com a demonstração do funcionamento do cronômetro de basquete na placa Nexys 1.

<https://youtu.be/cKieazU8pYo>

3 CONCLUSÃO

Com este trabalho, foi possível concluir com sucesso a criação, simulação e prototipação de um cronômetro de basquete completo usando a linguagem VHDL. A escolha de dividir o projeto em partes menores, separando a lógica central do cronômetro da parte que interage com a placa, foi uma estratégia muito importante. Isso ajudou a organizar a complexidade do circuito e facilitou bastante na hora de encontrar e testar erros.

O desenvolvimento de cada módulo ajudou a entender melhor como o hardware funciona na prática. A divisão do tempo em contadores separados para centésimos, segundos, minutos e quartos garantiu a precisão que o esporte exige. Além disso, a máquina de estados (FSM) cumpriu muito bem o papel de coordenar as pausas e as trocas de período. Um ponto forte do projeto foi a criação de proteções no código. Bons exemplos são o bloqueio automático quando o jogo acaba e o limite do carregamento manual, que se ajusta sozinho para as regras da FIBA (10 minutos) ou da NBA (12 minutos). Essas lógicas deixaram o circuito muito mais seguro contra comandos errados do usuário.

A fase de testes com os *testbenches* foi essencial para garantir que a contagem e a troca de estados estavam corretas no simulador, o que poupou muito tempo antes de ir para a placa física. Por fim, a prototipação na placa Nexys 1 foi a prova final do sucesso do projeto. Para o circuito interagir com o mundo real, foi preciso criar filtros para tirar o ruído dos botões (*debounce*) e mapear os pinos corretamente no arquivo UCF. No final, as chaves deslizantes, a formatação dos LEDs e a varredura visual do mostrador de sete segmentos funcionaram exatamente como o planejado. Isso confirmou que o cronômetro desenvolvido funcionou perfeitamente na prática.